From a problem to an unambiguous procedure

2026-11-01 | Day01/6 | Read 7 min + practice 3 min

Original course and learning scope

Harvard University · CS50’s Introduction to Computer Science · CS50x · 2026 (year URL redirects to current syllabus)

Week 0

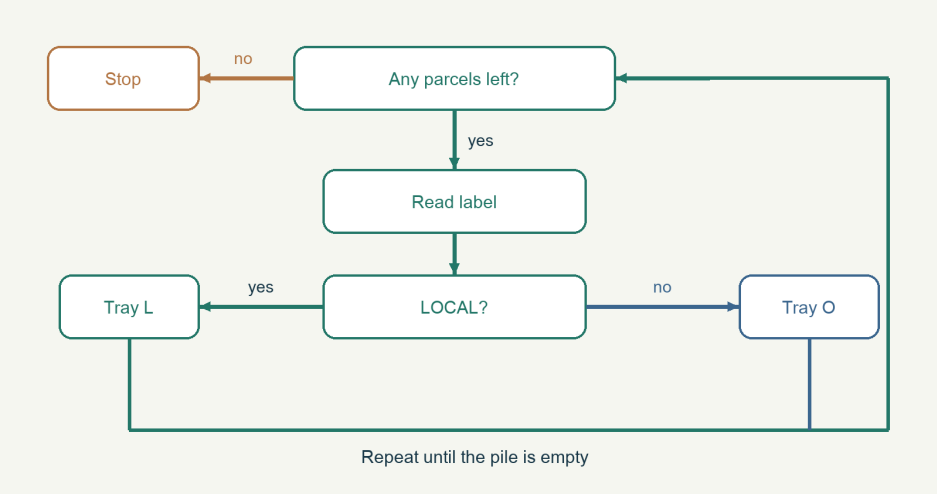
3/8 · Independent learning guide; not official course materials or completion credit.

Personal-time adaptation: units may be split; this is not the full original workload.

Light checks only; original assignments require a separate explicit choice.

Today’s place in the course

Today is a seven-minute reading followed by three minutes of light practice. We take one small part of u01, CS50x Week 0: expressing a problem as a procedure. The official Lecture 0 notes include problem solving, algorithms, pseudocode, and Scratch. [S2] You will not need to install anything. The example below is invented for this guide, and you can execute it with your finger on the page. By the end, explain an input, an output, and a reason that a small procedure finishes. That is a narrower, more useful target than “understand programming” in ten minutes.



An original parcel-sorting loop: check the pile, test a label, choose a tray, and repeat.

Original teaching diagram for this guide; not a CS50 slide or measured system.

First decide what counts as an answer

Imagine a volunteer sorting parcels into two trays. For this teaching example, each parcel has a destination label that is either LOCAL or OTHER. The desired output is two trays with every parcel placed according to its label. We assume the labels are readable and valid. Those assumptions matter: if a label is missing, the rules below do not yet say what to do. An input is the information or material the procedure receives. An output is the result required by the task. A procedure supplies steps linking them. "Sort carefully" states an intention but leaves decisions unspecified. "If the label is LOCAL, put the parcel in tray L; otherwise put it in tray O" gives the reader an executable decision under our assumptions.

Worked example: three parcels and an empty pile

Use this original pseudocode: while an unprocessed parcel remains, take one parcel; read its label; if the label is LOCAL, put it in L; otherwise put it in O. When no parcel remains, stop. Pseudocode lets us discuss the logic without committing to a particular programming language. Trace the labels OTHER, LOCAL, OTHER. At first both trays are empty and three parcels remain. After the first step, O contains one parcel and two remain. After the second, L contains one and one remains. After the third, O contains two, L contains one, and none remain. The loop test now fails, so the procedure finishes. Each successful pass removes exactly one item from the unprocessed pile. Now try zero parcels. The loop test fails immediately. Both trays stay empty, which is a sensible result for an empty input. A rule saying "take the first parcel" before checking the pile would fail here. Boundary cases are small enough to reason about and often reveal a hidden assumption quickly.

Read the figure as a trace of decisions

The diagram starts with the unprocessed pile, checks whether an item remains, and branches on its label. Both tray paths return to the pile check. The exit branch ends when the pile is empty. Follow the arrows for just one parcel before trying all three; the drawing represents repeated control flow, not three separate copies of the instructions. The diagram is our own teaching illustration. It does not show a real warehouse system, timing measurement or original CS50 slide. A real service might need rules for damaged labels, duplicate items and new arrivals. We deliberately freeze those complications so that you can see the core decision. A model becomes useful when its assumptions are visible.

Three minutes: predict before checking

1. Which instruction removes the ambiguity in "sort carefully"? A. Work faster. B. Read the label and choose L for LOCAL, otherwise O. C. Think about both trays. 2. Start with labels LOCAL, OTHER, LOCAL. How many items end in each tray? A short mental trace is enough. 3. Name one condition under which our rules need extending. One phrase is sufficient; there is no request to write a program or a long explanation. If you have time left, point to the exact step where that condition becomes a problem.

Suggested answers and a useful distinction

For 1, choose B: it specifies an observable test and an action for each permitted result. For 2, L receives two parcels and O receives one. The answer is checked against each label, not inferred from the shape of the diagram. For 3, a missing label is a good example: our "otherwise" would accidentally treat it as OTHER unless we introduce validation or a separate review tray. Stopping and correctness are different questions. A procedure can finish after putting every item in the wrong tray. Our progress argument explains why it stops for a finite, fixed pile; the label rule explains why each placement matches the specification. Keep both explanations available.

Next: express the same idea in blocks

Tomorrow stays in u01 and looks at decisions and repetition through Scratch. Bring the idea of a condition that can be true or false, plus a step that makes progress. You do not need to complete the official Scratch project today. The course context comes from official materials [S1][S2]. Our parcel example is not an attributed lecture demonstration. The 2026 links currently redirect to the main course pages; optional lecture viewing takes additional time beyond today's ten minutes.

Sources and original locations

S1 CS50x 2026 —Requested 2026 course URL redirects to https://cs50.harvard.edu/x/; official course context, week sequence and wider workflow, not a frozen archive.

Harvard University · CS50 | Published: 2026 edition label; page undated | Checked: 2026-11-01

S2 Lecture 0 —CS50x 2026 —Requested 2026 notes redirect to https://cs50.harvard.edu/x/notes/0/; verified topic headings and introductory concepts. Parcel example is original to this guide.

Harvard University · CS50 | Published: 2026 edition label; page undated | Checked: 2026-11-01

S3 CS50x 2026 syllabus —Original-course location: Syllabus; navigation Weeks 0–10; Artificial Intelligence; Expectations

Harvard University | Published: 2026 (year URL redirects to current syllabus) | Checked: 2026-11-01